

// **WAYSCONF 2026 · MASTERCLASS · 90 MINUTES**

---

# Build an AI that checks and fixes its own work

Agentic loops in practice.

Szymon Paluch · CTO, Susteen

ROOM-PM · 16 September 2026

// 0:00 – 0:08

---

# The case

# Most AI demos stop here

A prompt goes in. Something impressive comes out. Everyone claps.

Then someone asks a question the demo was not built to survive:

*Does it match the design?*

*Is the contrast legal?*

*Does it work on a phone?*

*Is that the actual copy, or copy it invented?*

no answer

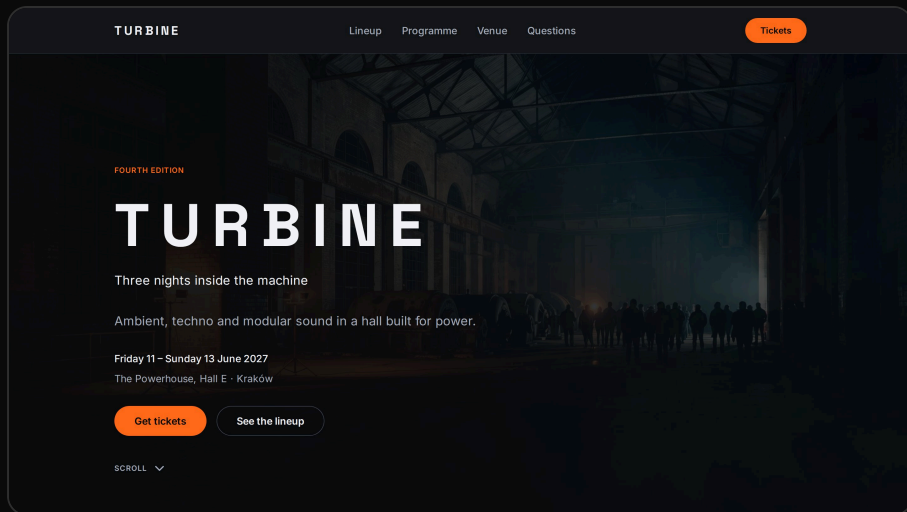
---

# This is a talk about the word *no*

A loop is only worth building if something in it can refuse  
the work without asking a language model for an opinion.

---

# What you will have at 16:25



[turbine-festival.netlify.app](https://turbine-festival.netlify.app) — open it, it is a real address

Built from a Figma file by an agent you instructed, checked by a program that agent wrote and is not allowed to change, fixed when it failed, and deployed to a URL you can send to someone.

## YOU LEAVE WITH

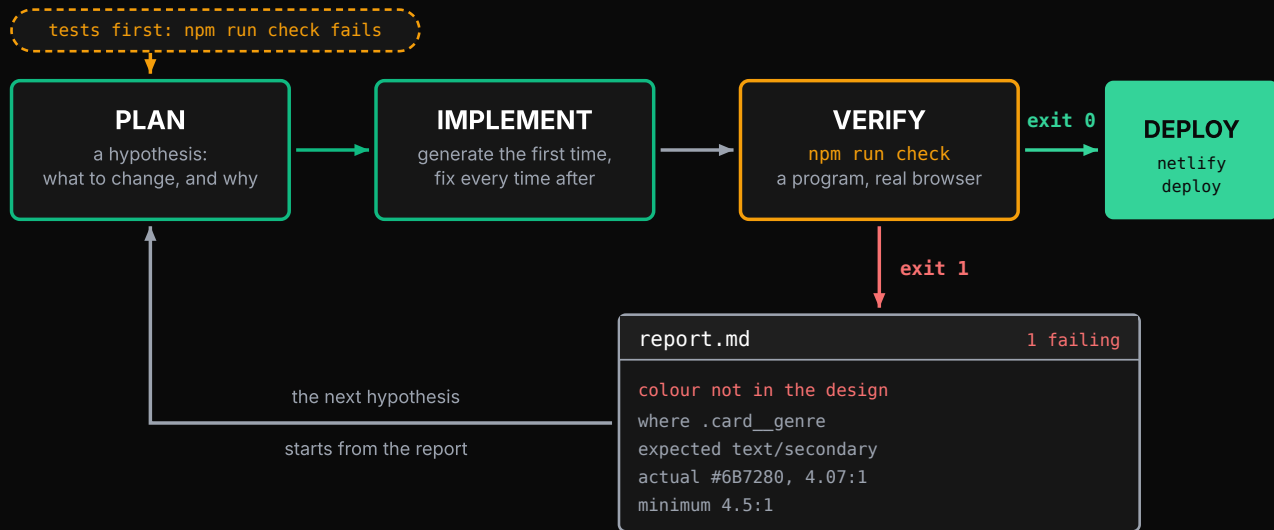
A URL, live on the internet. A folder: the page, and the checker that judged it. Eight prompts that work on Monday, on something that is not a festival.

```
exit 0
```

# The loop

## The loop

The check is written first and fails. Then plan, implement, verify, until it passes.



The way back carries a file. Nothing on it asks the model whether the work is good.

# Setup

**An agent running in an empty folder**

# The terminal, in one slide

## MACOS

`Cmd` + `Space`, type **Terminal**,  
`Enter`.

Paste with `Cmd` + `V`

## WINDOWS

`Win`, type **PowerShell**, open  
**Windows PowerShell**. Not  
Command Prompt.

Paste with `Ctrl` + `V`

## LINUX

`Ctrl` + `Alt` + `T`, or search for  
**Terminal**.

Paste with `Ctrl` + `Shift` + `V`

```
pwd           # where am I
ls            # what is here
cd turbine    # go in there
claude        # or: codex
```



No feedback while it works reads as "broken" and is almost always **"busy"**. You know a command has finished when the cursor line comes back.

**Esc** stops the agent. **Ctrl+C** twice closes it:  
`claude --continue` brings it back.

# You need two things

## THE DESIGN

`turbine.fig`, downloaded from the page — the file your agent reads. The same page opens the design in Figma, to look at.

## THE PROMPTS

[waysconf.szymonpaluch.com](https://waysconf.szymonpaluch.com)

Eight of them, with a copy button on each.

## AND AN EMPTY FOLDER

```
cd turbine      # made at home
claude          # or: codex
```



No `turbine` folder? Make one: `mkdir turbine`, then `cd turbine`. Prompt 01 creates the GitHub repository for you.

Agent up? Paste **prompt 01** now.

# 01 — Start

PROMPT 01

Copy

[Full text ↗](#)

Set up a new website project in this folder.

First make sure this folder is a git repository that pushes to GitHub, because every step after this one commits its work and pushes it.

1. Run `gh auth status`. If I am not signed in, stop and tell me.
2. Run `gh repo view turbine`. If a repository called turbine already exists on my account, make this folder a git repository whose origin remote is that repository, and pull whatever is already in it.

... 20 more lines. Copy takes the whole prompt.

No plan mode for this one. It checks your GitHub repository — **and creates it if you do not have one** — then makes the project, starts the development server, commits and pushes.

Every prompt is on the page with its own Copy button. Nothing here is typed by hand.

---

# Three of the eight start with a plan

|           | WHAT YOU DO                                                                      | CLAUDE CODE                                      | CODEx                   |
|-----------|----------------------------------------------------------------------------------|--------------------------------------------------|-------------------------|
| Plan      | <b>02, 03 and 04 only.</b> Switch to plan mode, paste, read the plan, approve it | <code>Shift+Tab</code> until <i>plan mode on</i> | <code>/plan</code>      |
| Implement | The agent builds. <b>01, 05, 06 and 07</b> start here: paste, and let them run   |                                                  |                         |
| Commit    | The agent commits the step and pushes it to your repository                      | written into the prompt                          | written into the prompt |
| Clear     | Empty the context before the next prompt                                         | <code>/clear</code>                              | <code>/clear</code>     |

Plan mode is for the three steps that build something new — the decoder, the checker, the page. The others have nothing to decide: they run a check, they deploy, they attack.

# Read the plan before it writes the code

Plan mode is the cheapest correction in the entire loop. A wrong assumption costs one sentence to fix here, and twenty minutes of generated code to fix later.

```
# Claude Code: Shift+Tab until the footer says "plan mode on"  
# Codex: /plan
```



Then read it. Actually read it. The plan is where you find out it misunderstood the brief.

---

// 0:13 — 0:38 • PROMPT 02 RUNS WHILE WE TALK

---

# Design in, not screenshots in

# Plan mode, then prompt 02

Put `turbine.fig` in `design/` inside your project. Switch to **plan mode**, paste **prompt 02**, read the plan, approve it.

## WHAT IT BUILDS

A decoder in your project: `npm run design`. It reads the `.fig` and writes the design as JSON in `design/data/`: sections, colours, typography, layout, components, copy, images.

The checker and the page both read those files. Nobody decodes or exports the design again.

## WHAT YOU DO

Approve the plan, then wait ten to fifteen minutes. Let it use the network if it asks.

It writes no page. It stops with a list and its open questions (point 6). Answer them; it commits and pushes. Then `/clear`.

# 02 — Look at the design

PROMPT 02

Copy

[Full text ↗](#)

Plan first. Research what you need, then show me a plan and wait for my approval. Do not create or change any file until I approve it.

Inside this project there is a folder called design with a .fig file in it. That is the Figma file itself, as Figma saves it. It is not a picture: it is the whole design as data – every text, colour, variable, component and photo.

Work from that one file and nothing else. Do not search this computer for the design, a specification, reference images or anything else about this project: there is nothing else,

... 73 more lines. Copy takes the whole prompt; a second part follows it on the page.

Read the plan for three things: it is a script `npm run design` runs, it writes JSON into `design/data`, it writes no page code.

# Two ways to hand over a design

## A SCREENSHOT

The agent looks at pixels and infers.

The orange becomes *an* orange.

The spacing becomes *some* spacing.

It looks roughly right.

token gate fails

## THE FILE ITSELF

Colours arrive as values. Spacing arrives as numbers.

```
TURBINE / Color
accent/sodium    #FF6A1A
text/secondary   #A7AEBB
text/muted       #6B7280
```



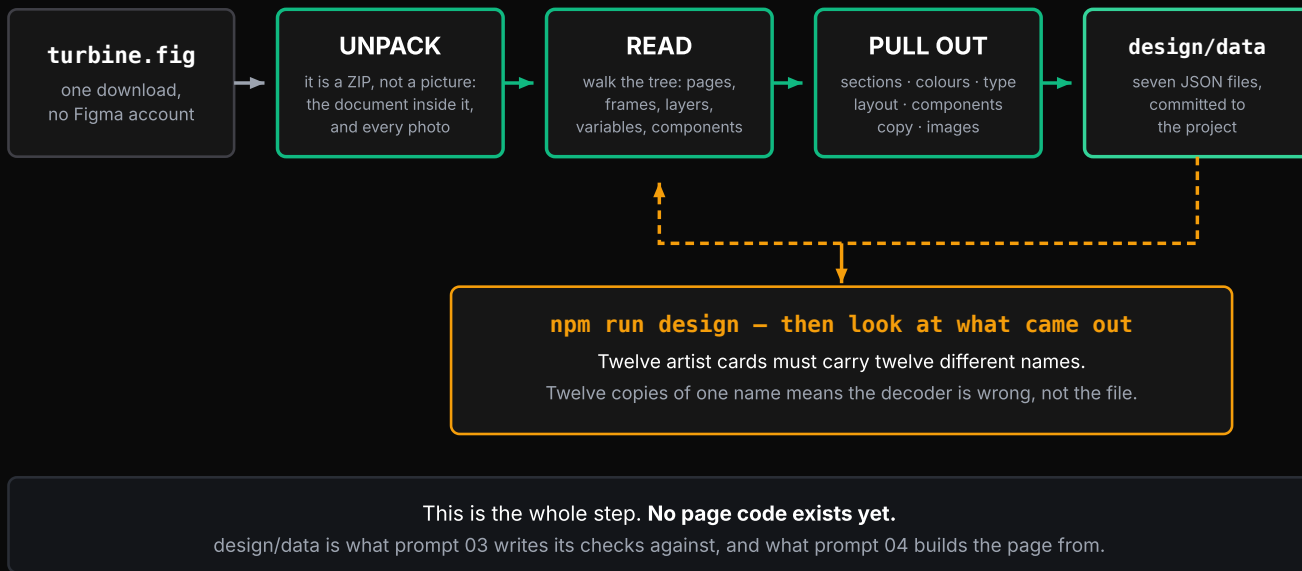
Nothing is guessed, so nothing is guessed wrong.

token gate passes

# The file, and what happens to it

## What it is doing while you listen

Prompt 02: one file in, seven JSON files out — and the agent checks its own output before it stops.



Your agent has never seen a **.fig** . It is writing a program to read one, right now, while I talk.

# The verifier

Written before the page

# Why a fresh context beats a long one

## ONE LONG CONVERSATION

Forty minutes of history.

Three abandoned approaches still in context.

The original brief, sixty thousand tokens ago.

Reasoning gets worse as it fills.

## A FRESH CONTEXT EACH PASS

Reads the report — the current failures.

Reads `notes.md` — what was already tried, and what it cost.

Reads design/data.

Notes on disk beat memory in context.

---

# Plan mode, then prompt 03

Answer 02's open questions first. Then `/clear`, switch to **plan mode**, paste **prompt 03**.

Your agent plans and writes `npm run check` from `design/data/`, before any page exists, and runs it on the empty project.

it must be red

Twenty minutes in the trial. A check that passes on an empty page is not checking anything. It commits; then `/clear`.

---

# 03 — Write the checker first

PROMPT 03

Copy

[Full text ↗](#)

Plan first. Research what you need, then show me a plan and wait for my approval. Do not create or change any file until I approve it.

Now write a program that checks the page against the design, before the page exists.

It must be a program, not an opinion. It runs, it looks at the real page in a real browser, and it exits with code 0 if everything is right and a non-zero code if anything is wrong. It never asks a language model anything, it never asks me anything, and it gives the same answer twice on the same page.

... 48 more lines. Copy takes the whole prompt.

The rule at the end of this prompt is the one that matters: **never change the checker to make a check pass.**

# Tests first

## RED

The check is written from the design data and run on the empty project.

It fails. The report lists everything the page does not have yet.

prompt 03

## GREEN

The page is built from the design data, then fixed against that report until the check passes.

prompts 04 and 05

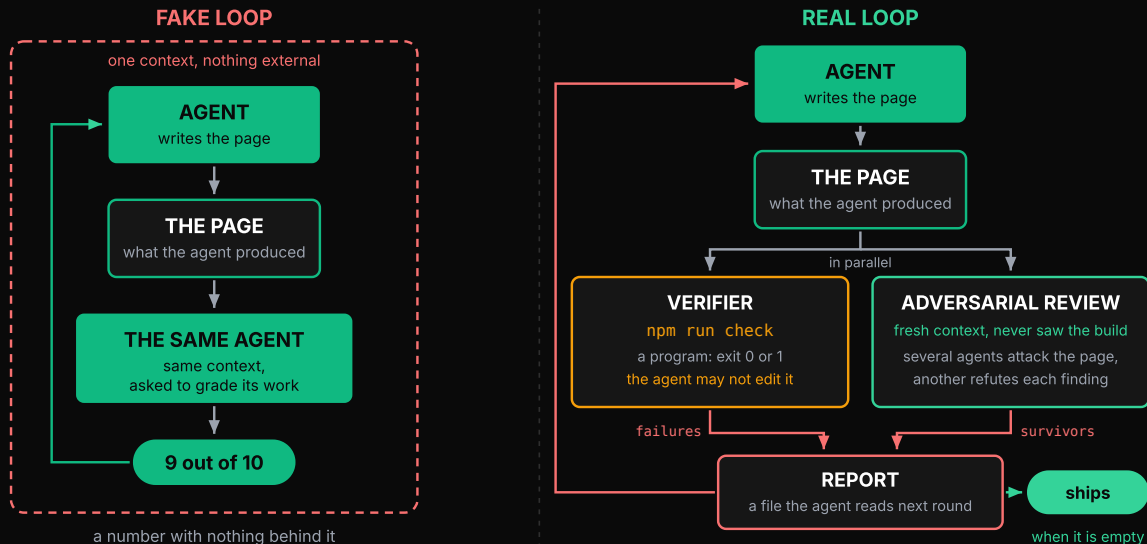
A check written after the page describes the page. A check written first can only describe the design.

---

# A real loop and a fake one

## Fake loop, real loop

An agent can check work. Not the same agent, in the same context, that did the work.



The program decides what can be measured. The fresh-context review looks for what cannot.  
Neither replaces the other, and neither is the agent grading itself.

# Who checks the work

## WHO CHECKS

## GOOD FOR

The same agent, same context

Nothing. It repeats its own reasoning

A program: `npm run check`

Anything with an exact answer: sections, colours, copy, contrast

Fresh-context agents, told to attack

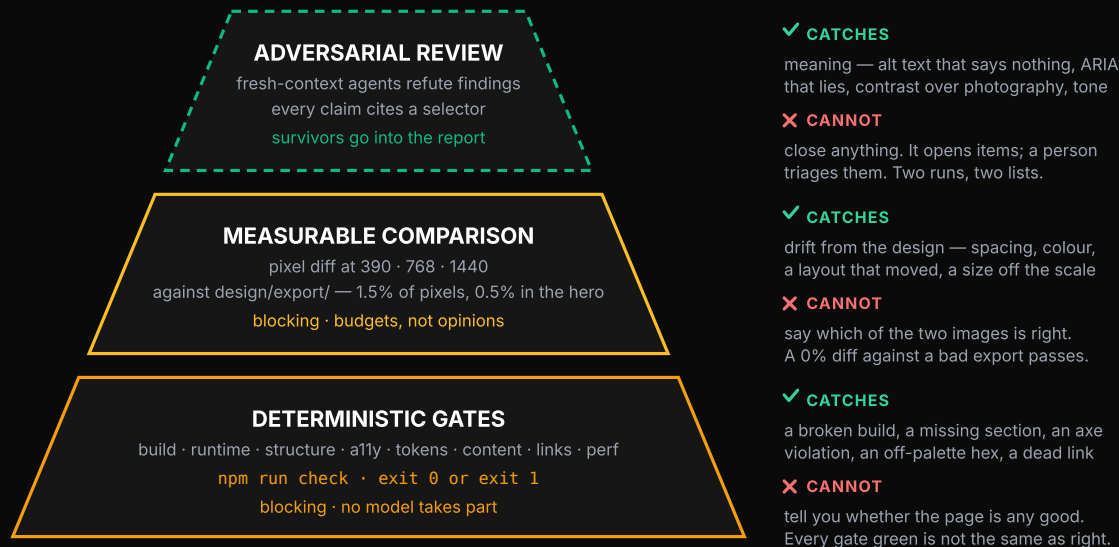
What a program cannot measure: reading order, alt text, text over photos, tone

Run the last two **side by side**. One report. It ships when the report is empty.

# Three tiers of checking

## Three tiers of verification

The bottom tier is the widest, the cheapest and the only kind that can be trusted to say no on its own.



The build gate runs first and alone; the seven browser gates then run together, and Lighthouse last.

Only the bottom two tiers can fail the build. Adversarial review runs alongside npm run check; what survives goes into the report.

Automated accessibility rules reach roughly 30 to 40 per cent of WCAG — a widely cited estimate, not a measurement of this page.

# The reference site: ten gates, no opinions

| GATE                    | WHAT IT DECIDES                                          |
|-------------------------|----------------------------------------------------------|
| Build                   | It compiles, no type errors, one page out                |
| Runtime                 | No console errors, no failed requests                    |
| Structure               | Sections, headings, landmarks, skip link, ids            |
| Accessibility           | axe clean at three widths, focus, keyboard, motion       |
| Design tokens           | Every colour and size came from the design               |
| Visual fidelity         | Pixels match the design, within a stated budget          |
| Content                 | The real copy, all twelve artists, no placeholders       |
| Links                   | Nothing points at nothing                                |
| Lighthouse              | Performance, a11y, best practices, SEO, and five budgets |
| <b>Text over images</b> | <b>The contrast axe refuses to judge</b>                 |

# The failure report is the interface

```
**3. [check 07] a colour on the page is not in the design**
```

- Where: the supporting line inside each card in the lineup section
- Expected: the design's "text / secondary"
- Actual: the design's "text / muted" – measured 4.07:1 against the page background,  
and body text needs 4.5:1
- Hint: muted is defined for legal and footer text only



No model wrote that. A program measured it and printed it.

The agent's next iteration starts by reading this file.

---

# Build the page

# Your turn — 15 minutes

## 1. PLAN MODE, PROMPT 04

It plans the page from `design/data/`, builds all of it, and runs the check once. Still red, with fewer failures.

## 2. REVIEW IT

Beside Figma, one section at a time. Say what is wrong in plain words. Then it commits, and you `/clear`.

Falling behind cannot hurt you.

Stop where you are. Leave whatever is unfinished. I want to move on.

Prompts 02 to 05 build on each other. Behind at the rescue time? Take the pack from the page and go on.

# 04 — Build it

PROMPT 04

Copy

[Full text ↗](#)

Plan first. Research what you need, then show me a plan and wait for my approval. Do not create or change any file until I approve it.

Build the page now, from the design. Read design/data and notes.md; do not decode the .fig again.

Your plan must list the sections in the order of design/data/sections.json, and say which colours, text styles, layout values, copy and images each one uses.

... 33 more lines. Copy takes the whole prompt; a second part follows it on the page.

It builds the page from `design/data`, not from a screenshot and not from taste. Then it runs the check once and stops.

---

// 1:13 — 1:21 • THE LOOP RUNS WHILE WE TALK

---

# The loop, and workflows

# Start your loop now, then listen

`/clear`, paste it, then leave it alone. I will talk over it; when it stops, it commits.

PROMPT 05

Copy

[Full text ↗](#)

Read notes.md. Then run `npm run check`.

The items under "Design review" in notes.md are problems I found by looking at the page. Treat the ones that design/data supports like failures in the report and fix them too. For each one design/data does not support, write one line into notes.md saying so, and leave it.

If `npm run check` exits 0 and no Design review item that design/data supports is left, commit everything with a message that says the check passes, push, and tell me we are finished.

... 25 more lines. Copy takes the whole prompt; a second part follows it on the page.

Both agents also have a goal command — `/goal npm run check exits 0`. A separate check judges it after every turn and the agent keeps going until it is met; the exit code still decides. Stop early with `/goal clear`.

# The whole loop is four sentences

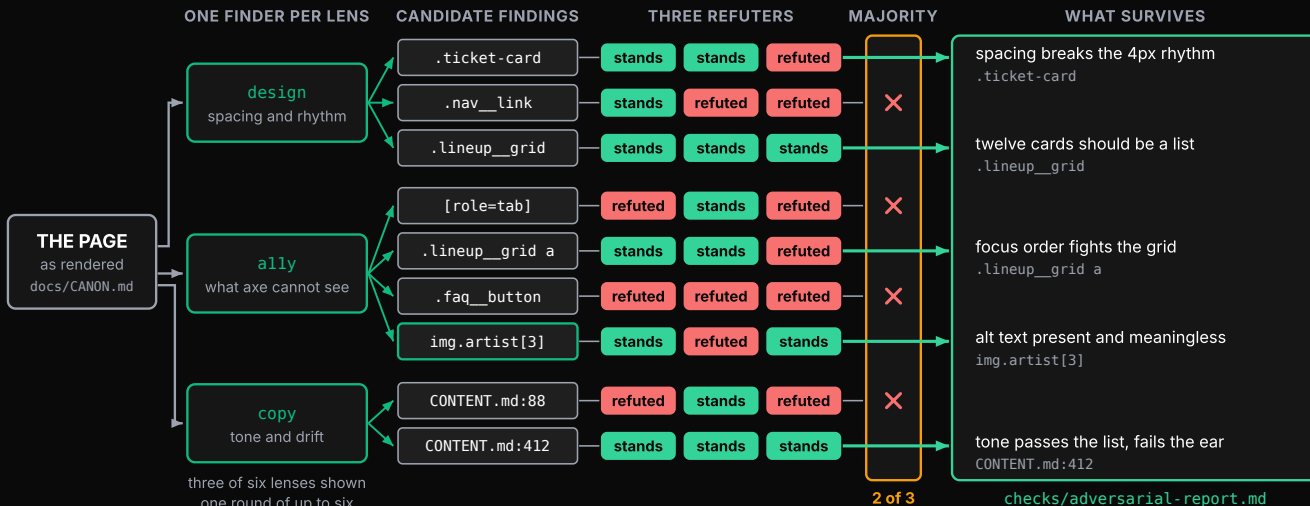
|                                |                                                                                                          |
|--------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Check first</b>             | Run the check <i>before</i> you fix anything. An agent asked to improve passing work will find a reason. |
| <b>Hand back the report</b>    | Unchanged. The file a program wrote, not your summary of it.                                             |
| <b>Never touch the checker</b> | The moment the thing being judged can edit the judge, every green result afterwards means nothing.       |
| <b>Stop on a program</b>       | Not when it feels finished. When something exits 0.                                                      |

No script. Nothing to install. You can say this to any agent, today.

# When one agent is not enough

## One finder per lens, three refuters per finding

The work fans out, gets argued with, and only the part that survives a majority reaches a person.



### Nothing here runs in lockstep.

Each candidate goes to three refuters whose only instruction is to argue that it is wrong, and reaches the filter when its own panel finishes. A lens that returns two findings does not hold up a lens that returns four.

The filter is arithmetic: two of three must stand. Every claim cites a selector or a file and line, or it is discarded unread.

Nothing on this page sets pass or fail. It opens items for a person to triage.

# Adversarial review

The instruction is not "review this".

The instruction is **"refute this, and default to refuted if you are unsure"**.

```
ultracode. ... inside a phase, spawn as many subagents as the work  
needs and run them in parallel, deciding how many from what you find
```

```
PHASE 07 – ATTACK
```

```
... for every candidate they find, a separate subagent whose only  
job is to argue against it from those three angles.
```



That is **prompt 08**: prompts 01–07 as one message, phase for phase, with the differences it lists. Take it home. Today, **prompt 07** in a new session, after the deploy, is the one-agent version.

---

# o8 — The same job, as a workflow

PROMPT 08

Copy

[Full text ↗](#)

ultracode. I want you to do a whole job in seven phases. Each phase is one of the seven workshop prompts, in the same order, with the same words and the same rules. Work through them on your own and do not skip ahead.

Use a dynamic workflow: inside a phase, spawn as many subagents as the work needs and run them in parallel, deciding how many from what you find rather than from a number I gave you — one per section, one per review lens, one per finding, whatever the phase calls for. Merge their results before you leave the phase.

... 367 more lines. Copy takes the whole prompt.

Everything you did today, in one message. Use it on Monday on something that is not a festival.

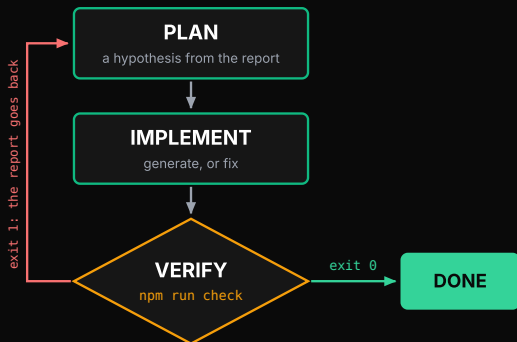
# Two shapes, and neither is a script

## Two shapes

A loop repeats until a condition holds. A workflow runs phases in order, and a phase can branch.

### THE LOOP

repeat until a condition holds



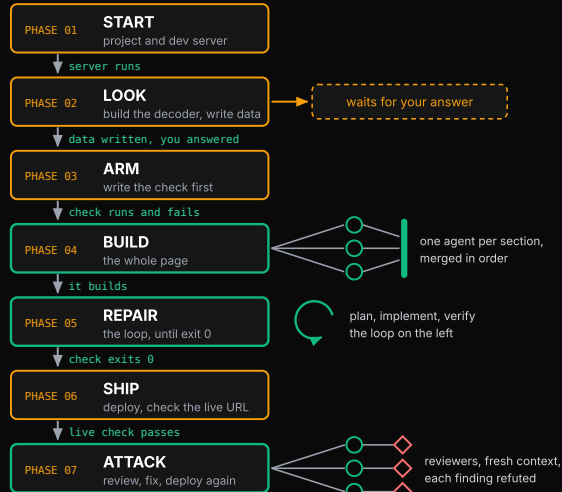
You define the exit condition.

Check written first. Then plan, implement, verify, until exit 0.

```
prompt 05 · /goal npm run check exits 0
```

### THE WORKFLOW

phases in order, a bar between each



You define the phases, what branches inside them, and the bar between.

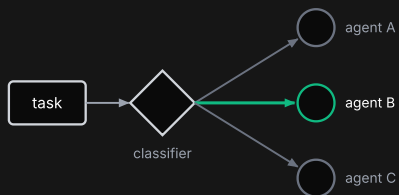
```
prompt 08
```

They nest: phase 05 of the workflow is the loop. Both are written as instructions, not code.

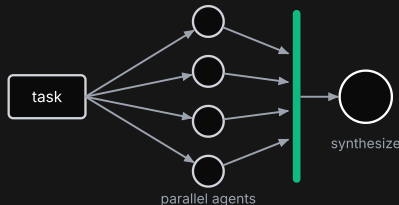
# Six workflow patterns

## Six workflow patterns

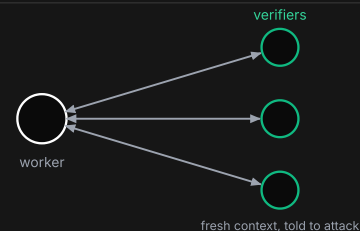
### 1 Classify and act



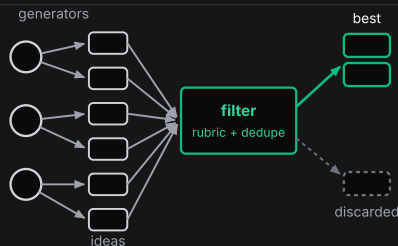
### 2 Fan out and synthesize



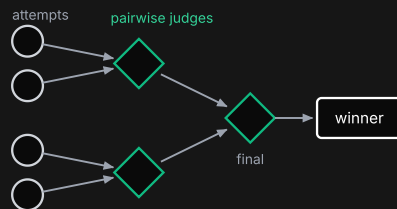
### 3 Adversarial verification



### 4 Generate and filter



### 5 Tournament



### 6 Loop until done



# Ship, then break it

# Deploy it, then try to break it

`/clear`, then **prompt 06**. It builds, creates your Netlify site without asking questions, deploys, and checks that the page the server returns is the page that passed. Take the URL it prints.

**Prompt 07 is tonight's.** An agent that never saw the build tries to break the page. You pick the findings that are real; it fixes them, the check still passes, and it deploys again. Half an hour, and you do not have to watch it.

Put your URL on the board. We are going to look at all of them.

---

# o6 — Ship it

PROMPT 06

Copy

[Full text ↗](#)

Put this on the internet.

Build the site, then deploy it to Netlify with the Netlify command line. It is installed and I am signed in; if it says I am not, tell me what to do rather than doing it silently.

This project has no Netlify site yet. Create it and deploy in one command, without interactive questions: `netlify deploy --prod --dir=dist --site-name turbine-<my GitHub username>`. Find my username with `gh api user --jq .login`. If that name is taken, add a short suffix and run it again.

... 12 more lines. Copy takes the whole prompt.

The last line is the one that makes it a check and not a hope: run the same checker **against the address the server returns**.

# 07 — Try to break it

PROMPT 07

Copy

[Full text ↗](#)

Whatever npm run check says right now, try to prove the page is wrong.

You did not build this page. Judge it only by what a browser shows and what is in this project, not by anything said earlier in this conversation.

Look at the page on this machine. If the development server is not running, start it yourself and give me the address.

Your job now is to attack, not to defend and not to fix. Find five things that

... 31 more lines. Copy takes the whole prompt; a second part follows it on the page.

Tonight, not now. Thirty minutes, a context that never saw the build, and a brief to **attack**.

# What this does not do

# Take this to work on Monday

Do not start with a whole landing page. Start with one gate.

1. Pick the check your team already does by hand and resents.
2. Make it a program that exits 0 or 1.
3. Put the failure output somewhere an agent can read.
4. Only then put an agent in front of it.

The loop is the easy part. The oracle is the work.

---

// EVERYTHING FROM TODAY

---

# TURBINE

waysconf.szymonpaluch.com

The eight prompts, in English and Polish · the `.fig` · these slides  
and the repository, with the checker and the evidence: `github.com/hculap/waysconf-2026-agentic-loops`

Questions. And then go and put your URL on the board.